

Real-Time E-Commerce Lakehouse & Zero-ETL Analytics Architecture (Pub/Sub)

INDEX / TABLE OF CONTENTS

- 1. Project Overview**
- 2. Tools & Services Used**
- 3. End-to-End Data Flow**
- 4. Real-Time E-Commerce Data Pipeline Architecture**
- 5. Pipeline Build & Execution Instructions**
 - 5.1 Create Cloud Storage Buckets & Folders**
 - 5.2 Set Up Pub/Sub & Storage Permissions**
 - 5.3 Provision Cloud Spanner Instance & Table**
 - 5.4 Run the Stream Publisher**
 - 5.5 Run Dataproc Serverless PySpark**
 - 5.6 Create BigQuery External Connection & Dataset**
 - 5.7 Execute & Schedule Federated Analytics**
 - 5.8 Pipeline Data Verification**

Aim:

An event-driven, serverless streaming architecture designed to ingest real-time JSON order streams via Cloud Pub/Sub, transform and deduplicate payloads using Dataproc Serverless PySpark, and concurrently load dual destinations: a Medallion GCS Lakehouse (Bronze JSON to Silver Parquet) and an OLTP database (Cloud Spanner), enabling zero-copy federated analytics and scheduled aggregations in BigQuery via EXTERNAL_QUERY.

Tools & Services Used

1.Cloud Pub/Sub (*The Conveyor Belt*): Ingests streaming order events in real time and routes raw payloads directly into cloud storage using a native Cloud Storage (GCS) write subscription.

2.Cloud Storage / GCS (*The Digital Storage Lockers*):

- Bronze Bucket (intricate-tempo-raw-ecom): Raw landing storage holding original JSON records exactly as received.
- Silver Bucket (intricate-tempo-cleaned-ecom): Cleaned, deduplicated, and compressed Parquet storage for long-term historical lakehouse analytics.

3.Dataproc Serverless / PySpark (*The Automated Data Washer*): Fully managed, serverless compute engine running clean_ecom.py to explode nested product arrays into individual line items, cast data types, and eliminate duplicates.

4.Cloud Spanner (*The Operational Cash Register*): High-speed, globally consistent OLTP relational database hosting the EcomOrders table (Primary Key: order_id, product_id) for live transactional access.

5.BigQuery External Connection (*The Federated Telescope*): Zero-ETL bridge (spannerecom-conn) that enables direct, live querying of Cloud Spanner from BigQuery using EXTERNAL_QUERY without copying data.

6.BigQuery Studio & Scheduler (*The Automated Report Generator*): Executes scheduled SQL queries daily at midnight to aggregate live metrics and append results to a persistent summary table in asia-south1.

QUICK SUMMARY:

- Pub/Sub = Real-time messenger
- GCS = Storage buckets (Raw vs Clean)
- Dataproc = Scalable data washer
- Spanner = High-speed live database
- BigQuery Connection = Direct view without copying data

- BigQuery Scheduler = Daily automatic report generator

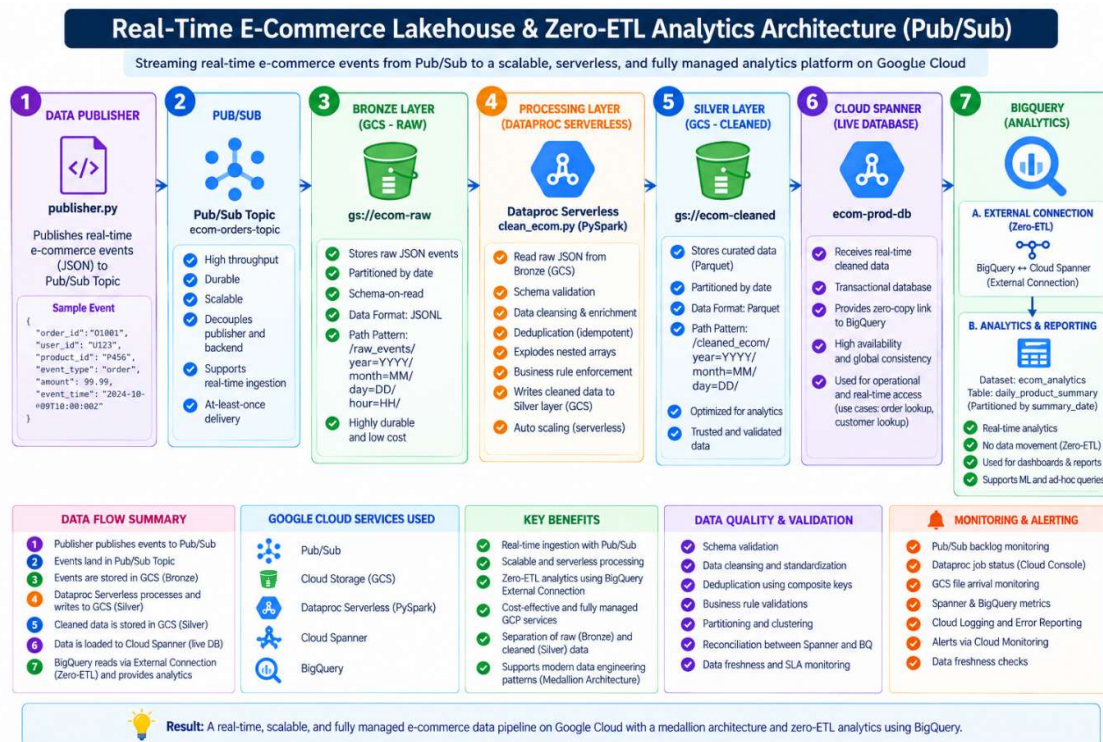
3. End-to-End Data Flow:

1. **Publisher (publisher.py):** Generates and publishes streaming e-commerce order payloads in JSON format to a Cloud Pub/Sub Topic.
2. **Raw Ingestion:** A GCS Pub/Sub subscription writes the raw streams directly into the Bronze GCS Bucket.
3. **ETL & Processing:** Dataproc Serverless (PySpark) reads the raw JSONs, normalizes nested arrays into line items, enforces data types, deduplicates records, and splits the data pipeline into two targets:
 - Path A (Data Lake): Writes compressed Parquet files into the Silver GCS Bucket.
 - Path B (OLTP Store): Appends flattened relational records directly into the Cloud Spanner table (EcomOrders).
4. **Federated Query Layer:** BigQuery queries Cloud Spanner in real time using the spanner-ecom-conn external connection via the EXTERNAL_QUERY function.
5. **Scheduled Aggregations:** BigQuery Data Transfer Service runs daily scheduled jobs to aggregate order data and write outputs to a daily summary analytical table.

4. Real-Time E-Commerce Data Pipeline Architecture:

- 1. **Ingestion Layer:** The publisher.py script generates real-time e-commerce order streams in raw JSON format and sends them to a Cloud Pub/Sub Topic.
- 2. **Landing (Bronze Layer):** A GCS Subscription on Pub/Sub automatically dumps the unparsed JSON streams into the Raw GCS Bucket (intricate-tempo-raw-ecom).
- 3. **Distributed Compute & ETL:** Dataproc Serverless (PySpark) picks up raw JSON files using the GCS SDK, normalizes nested item arrays, casts data types, and deduplicates records. It then branches into two output paths:
 - Storage Path (Silver Layer):** Saves cleaned, compressed Parquet files into the Cleaned GCS Bucket (intricate-tempo-cleaned-ecom) for long-term historical analytics.
 - **Operational Path (Gold / OLTP):** Writes live, operational order records directly into the Cloud Spanner table (EcomOrders).

- **4. Federated Query & Zero-ETL:** A BigQuery External Connection (spanner-ecomconn) uses EXTERNAL_QUERY to bridge directly to Cloud Spanner, allowing BigQuery Studio to query live transactional data in real time without moving or duplicating data.
- **5. Automated Analytics:** BigQuery Scheduler executes automated daily aggregation queries to process the live federated data and append metrics to a persistent Daily Summary Table.



5. Pipeline Build & Execution Instructions:

5.1 Step 1: Create Cloud Storage Buckets & Folders

Set up Bronze and Silver storage layers.

- In the GCP Console, navigate to **Cloud Storage > Buckets** and click **+ Create**.
- **Bronze Bucket (Raw Data):**
 - **Name:** pub-sub-rawdata

- **Location type:** Region → asia-south1 (Mumbai)
- ○ Leave other settings as default and click **Create**.
- **Silver Bucket (Cleaned Data & Scripts):**
 - **Name:** pub-sub-clean-data
 - **Location type:** Region → asia-south1 (Mumbai)
 - Click **Create**.
- **Inside pub-sub-clean-data, create two folders:**
 - scripts/
 - parquet/

5.2 Step 2: Set Up Pub/Sub & Storage Permissions *Configure message broker and bucket-level IAM.*

- Go to **Pub/Sub > Topics** → Click + **Create Topic**.
 - **Topic ID:** ecom-raw-topic ○ Click **Create**.
- Open ecom-raw-topic → Go to the **Subscriptions** tab → Click + **Create Subscription**.
 - **Subscription ID:** ecom-raw-gcs-sub
 - **Delivery type:** Select **Write to Cloud Storage**.
 - **Cloud Storage bucket:** Select pub-sub-rawdata.
 - **Filename prefix:** raw_order_
 - **Output format:** JSON.
 - Click **Create**.
- **Grant IAM Permissions:**
 - Go to **Cloud Storage > Buckets > pub-sub-rawdata > Permissions** tab.
 - Click + **Grant access** (or Add Principals).
 - **New principal:** service-54903645206@gcp-sapubsub.iam.gserviceaccount.com ○
 - Assign two roles:

- Storage Legacy Bucket Reader
- Storage Object Creator
- Click **Save**.

5.3 Step 3: Provision Cloud Spanner Instance & Table

Provision transactional OLTP database and schema.

- Go to **Spanner > Instances** → Click + **Create Instance**.
 - **Instance name / ID:** ecom-spanner-instance
 - **Configuration:** Regional → asia-south1 (Mumbai)
 - Click **Create**.
- Inside the instance, click + **Create Database:** ○ **Database name:** ecom-db ○ **Dialect:** Google Standard SQL
- In the DDL definition window, execute: SQL Query provided in code documentation
- Click **Create**.

5.4 Step 4: Run the Stream Publisher (publisher.py)

Generate mock streaming events into Pub/Sub.

- Save the following code locally or in Cloud Shell as publisher.py:
Take it from the code documentation
- Run the script: `python3 publisher.py`
- Check **Cloud Storage > pub-sub-rawdata** to verify raw_order_... JSON files have landed.

5.5 Step 5: Run Dataproc Serverless PySpark (clean_ecom.py)

Cleanse, explode nested arrays, and write to dual sinks.

Create clean_ecom.py containing the GCS Python SDK ingestion fix (bypassing Hadoop URI colon issues):

Take it from the code documentation

- Upload the script to GCS: `gcloud storage cp clean_ecom.py gs://pub-sub-clean-data/scripts/clean_ecom.py`
- Go to **Dataproc > Serverless > Batches** → Click + **Submit Batch:**

- **Batch ID:** clean-ecom-final-run-01 ○
- Region:** asia-south1
- **Batch type:** PySpark
- **Runtime version:** 2.3 LTS (Spark 3.5, Java 17, Scala 2.13)
- **Main python file:** gs://pub-sub-clean-data/scripts/clean_ecom.py
- **Properties (Add 2):**
 - spark.jars.packages=com.google.cloud.spark.spanner:spark-3.5spanner:1.4.0
 - spark.spanner.projectId = snappy-mapper-498509-e0
- Click **Submit** and wait for **Succeeded**.

5.6 Step 6: Create BigQuery External Connection & Dataset

Establish live connection from BigQuery to Spanner.

- **Create BigQuery Connection:**
 - In BigQuery Studio, click + **Add Data > Connections to external data sources**.
 - **Connection type:** Cloud Spanner
 - **Connection ID:** spanner-ecom-conn
 - **Location type:** Region → asia-south1 (Mumbai)
 - **Database name:**

projects/snappy-mapper-498509-e0/instances/ecom-spanner-instance/databases/ecom-db

- Click **Create connection**.
- **Create BigQuery Dataset:**
 - In the Explorer pane, click the three dots (:) next to snappy-mapper-498509-e0 → **Create dataset**.
 - Dataset ID:** ecom_analytics
 - **Location:** Region → asia-south1 (Mumbai)

- Click **Create dataset**.

5.7 Step 7: Execute & Schedule Federated Analytics

Materialize initial table and configure recurring run.

- **Create and Populate the Target Table (CTAS):**

Run this in BigQuery Studio to create the table and insert the initial aggregated batch:

OPTIONAL:

```
CREATE TABLE IF NOT EXISTS `snappy-mapper-498509e0.ecom_analytics.daily_product_summary` AS
SELECT
  product_id,
  SUM(quantity) AS total_items_sold,
  COUNT(DISTINCT order_id) AS total_orders,
  CURRENT_TIMESTAMP() AS aggregated_at
FROM EXTERNAL_QUERY(
  "projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecomconn",
  "SELECT order_id, product_id, quantity FROM EcomOrders"
)
GROUP BY product_id;
```

- **Schedule the Incremental Job:**

Paste the append query in the editor:

SQL

```
INSERT INTO `snappy-mapper-498509-e0.ecom_analytics.daily_product_summary` (
  product_id,
  total_items_sold,
  total_orders,
  aggregated_at
)
SELECT
```

```

product_id,
SUM(quantity) AS total_items_sold,
COUNT(DISTINCT order_id) AS total_orders,
CURRENT_TIMESTAMP() AS aggregated_at
FROM EXTERNAL_QUERY(
  "projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecomconn",
  "SELECT order_id, product_id, quantity FROM EcomOrders"
)
GROUP BY product_id;

```

- Click **Schedule > Create new scheduled query**.
 - **Display name:** daily-spanner-sales-summary
 - **Frequency:** Daily
 - **Location:** asia-south1
 - Click **Save**.

5.8 Step 8: Pipeline Data Verification

Confirm data has loaded successfully across all layers.

- Run this SQL query in BigQuery Studio to verify data population in the final analytics summary table:

Take it from the code documentation